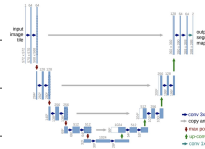


U-Net



- 핵심 아이디어

- ① 인코더의 피쳐맵을 디코더 피쳐맵에 Concat 하여 위치 정보 전달
- ② 데이터셋의 전처리, 변형(Deformation)을 이용해 데이터 증강
- ③ 테두리를 더 잘 붙일 수 있게 weight를 특정한 손실함수 사용

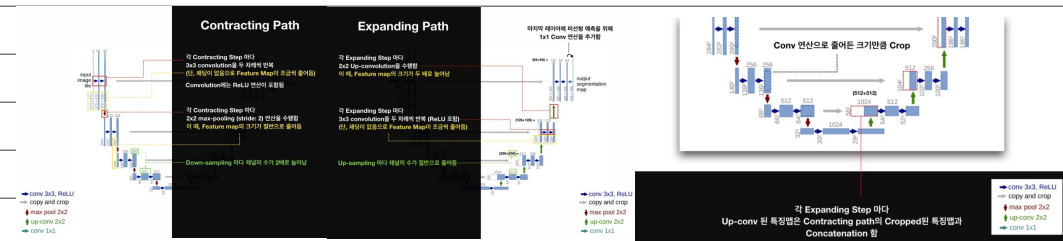
구조

- 일반적인 semantic segmentation

- Down-sampling
- Up-sampling

- U-Net의 구조

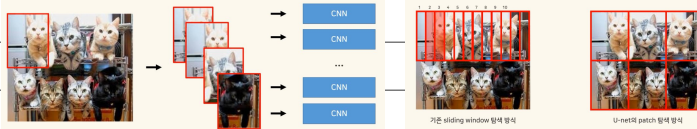
- Contracting Path = 점진적으로 넓은 범위의 이미지를 보면 의미정보 (Context Information) 축적 (Down-sampling)
- Expanding Path = Context Information을 여러번 up-sampling 하는 과정 (Output segmentation map의 결과 채널은 클래스개수)
- Skip Architecture = FCN과 비슷하게 중간에 나온 feature map을 사용하지만 U-Net은 같은 Level의 feature map끼리 묶는다.



Input Image에 적용한 기법

- 사용된 이유

- 기존의 이미지를 DNN에 넣기 위해서는 sliding window 탐색 기법을 사용하여 다시 훑어 보는 작업때문에 시간이 오래걸린다.
- 이를 해결하기 위해서 Patch 탐색을 제시 하였다. Patch 탐색은 지나왔던 부분은 잊고 새로운 것에 집중하는 탐색 방법이다.

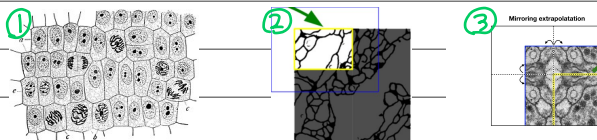


- Overlap-tile Strategy (중첩타일 전략), Mirroring Extrapolation (거울 확장법)

- ① U-Net 외곽선 (512x512)를 넣어 (388x388)크기의 Segmentation map이 출력된다.

➡ Convolution 연산에 Padding 없이 진행시켜서)

- ② 1번 방식때문에 missing data가 생겨서 해결하기 위해 사용한다.



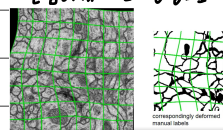
- ③ 그림 2와 같이 파랑색 상자안에 빈공간을 그림 3 처럼 거울에 반사된 형태로 채운다.

- ④ 이렇게 하면 zeroPadding 보다 Data Augmentation 효과를 줄수있다.

- Data Augmentation

- 눈물에는 데이터가 별로 없어 (30개) 데이터를 병행시켰다.
- ① shift ② Rotation ③ Gray value ④ Elastic Deformation

➡ Pixel이 랜덤하게 다른 방향으로 뒤돌리도록 변형하는 방식



학습

- 손실 함수(Loss func)를 픽셀마다 구한 Energy function의 총합으로 구현하였다.

Energy function

$$\text{Energy function} = \sum_{x \in \Omega} w(x) \cdot \log(p_{\lambda(x)}(x))$$

Cross-Entropy

클래스

Weightmap

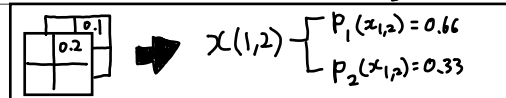
Softmax 결과값

- Soft max

- 기존의 Soft max는 $\frac{\text{결과값}}{\text{모든 결과값}}$ 으로 사용되었다
- 여기서는 아래처럼 각 채널별 Soft max 결과를 출력하고

$temp1 = \frac{(1,1) \text{ 위치의 결과값}}{1 \text{ 채널의 모든 결과값}}$	$\rightarrow$	<table border="1"><tr><td>0.1</td><td>0.2</td></tr><tr><td>0.3</td><td>0.4</td></tr></table> 채널 D	0.1	0.2	0.3	0.4
0.1	0.2					
0.3	0.4					
$temp2 = \frac{\text{"}}{2 \text{ 채널 "}}$	$\rightarrow$	<table border="1"><tr><td>0.3</td><td>0.1</td></tr><tr><td>0.2</td><td>0.4</td></tr></table> 채널 Q	0.3	0.1	0.2	0.4
0.3	0.1					
0.2	0.4					

- 아래 그림처럼 (x,y) 포인트에서 각 클래스별 Soft max를 만든다



- Cross-Entropy

- 예상한 확률과 실제정답 확률을 비교한다.

$$\text{정답}[1,0] \leftarrow - \sum_{\text{class}} P_{\text{class}}(x) \cdot \log Q_{\text{class}}(x) \rightarrow \text{예측}[0.9,0.2]$$

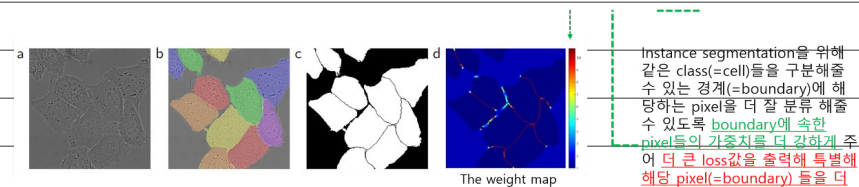
$$\begin{aligned} \text{정답이양변수} &\rightarrow -(1 \cdot \log 0.98 + 0 \cdot \log 0.02) & \text{정답이양다음} &\rightarrow -(1 \cdot \log 0.02 + 0 \cdot \log 0.98) \\ &= -\log 0.98 \approx 0.02 & &= -\log 0.02 \approx 1.69 \end{aligned}$$

- Weight map

- 경계선 픽셀에 더 큰 Loss를 주기 위해서 사용한다

The weight map equation = $w(x) = w_c(x) + w_0 \cdot \exp\left(-\frac{(d_1(x) + d_2(x))^2}{2\sigma^2}\right)$ $\rightarrow$ 해당 픽셀 ($x \in Z^2$) 의 중요도를 결정해주는 함수

$\rightarrow$ 식에 대한 자세한 설명은 다음 그림에서



$$\text{Energy function} = E = \sum_{x \in \Omega} w(x) \log(p_{\lambda(x)}(x))$$

특징

- 데이터의 양이 적음에도 전처리와 보충으로 극복했다.
- 특정 데이터에 한정된 기법이지만 데이터 도메인(특성)을 잘 파악하였다.